

Original

Simulated Annealing

Simulated Annealing (SA) is a randomized algorithm, which approximates the global optimum of a function. It's called a randomized algorithm, because it employs a certain amount of randomness in its search and thus its output can vary for the same input.

The problem

We are given a function $E(s)$, which calculates the energy of the state s . We are tasked with finding the state s_{best} at which $E(s)$ is minimized. **SA** is suited for problems where the states are discrete and $E(s)$ has multiple local minima. We'll take the example of the [Travelling Salesman Problem \(TSP\)](#).

Travelling Salesman Problem (TSP)

You are given a set of nodes in 2 dimensional space. Each node is characterised by its x and y coordinates. Your task is to find an ordering of the nodes, which will minimise the distance to be travelled when visiting these nodes in that order.

Motivation

Annealing is a metallurgical process, wherein a material is heated up and allowed to cool, in order to allow the atoms inside to rearrange themselves in an arrangement with minimal internal energy, which in turn causes the material to have different properties. The state is the arrangement of atoms and the internal energy is the function being minimised. We can think of the original state of the atoms, as a local minima for its internal energy. To make the material rearrange its atoms, we need to motivate it to go across a region where its internal energy is not minimised in order to reach the global minima. This motivation is given by heating the material to a higher temperature.

Simulated annealing, literally, simulates this process. We start off with some random state (material) and set a high temperature (heat it up). Now, the algorithm is ready to accept states which have a higher energy than the current state, as it is motivated by the high temperature. This prevents the algorithm from getting stuck inside local minimas and move towards the global minima. As time progresses, the algorithm cools down and refuses the states with higher energy and moves into the closest minima it has found.

The energy function $E(s)$

$E(s)$ is the function which needs to be minimised (or maximised). It maps every state to a real number. In the case of TSP, $E(s)$ returns the distance of travelling one full circle in the order of nodes in the state.

State

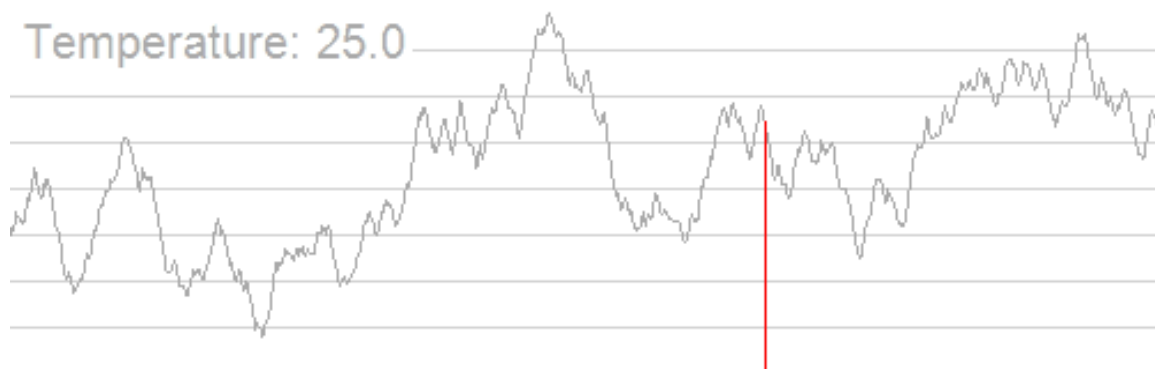
The state space is the domain of the energy function, $E(s)$, and a state is any element which belongs to the state space. In the case of TSP, all possible paths that we can take to visit all the nodes is the state space, and any single one of these paths can be considered as a state.

Neighbouring state

It is a state in the state space which is close to the previous state. This usually means that we can obtain the neighbouring state from the original state using a simple transform. In the case of the Travelling Salesman Problem, a neighbouring state is obtained by randomly choosing 2 nodes, and swapping their positions in the current state.

Algorithm

We start with a random state s . In every step, we choose a neighbouring state s_{next} of the current state s . If $E(s_{next}) < E(s)$, then we update $s = s_{next}$. Otherwise, we use a probability acceptance function $P(E(s), E(s_{next}), T)$ which decides whether we should move to s_{next} or stay at s . T here is the temperature, which is initially set to a high value and decays slowly with every step. The higher the temperature, the more likely it is to move to s_{next} . At the same time we also keep a track of the best state s_{best} across all iterations. Proceeding till convergence or time runs out.



A visual representation of simulated annealing, searching for the maxima of this function with multiple local maxima.

Temperature(T) and decay(u)

The temperature of the system quantifies the willingness of the algorithm to accept a state with a higher energy. The decay is a constant which quantifies the "cooling rate" of the algorithm. A slow cooling rate (larger u) is known to give better results.

Probability Acceptance Function(PAF)

$$P(E, E_{next}, T) = \begin{cases} \text{True} & \text{if } \mathcal{U}_{[0,1]} \leq \exp(-\frac{E_{next}-E}{T}) \\ \text{False} & \text{otherwise} \end{cases}$$

Here, $\mathcal{U}_{[0,1]}$ is a continuous uniform random value on $[0, 1]$. This function takes in the current state, the next state and the temperature, returning a boolean value, which tells our search whether it should move to s_{next} or stay at s . Note that for $E_{next} < E$, this function will always return True, otherwise it can still make the move with probability $\exp(-\frac{E_{next}-E}{T})$, which corresponds to the [Gibbs measure](#).

```
bool P(double E, double E_next, double T, mt19937 rng){
    double prob = exp(-(E_next-E)/T);
    if(prob > 1) return true;
    else{
        bernoulli_distribution d(prob);
        return d(rng);
    }
}
```

Code Template

```
class state {
public:
    state() {
        // Generate the initial state
    }
    state next() {
        state s_next;
        // Modify s_next to a random neighboring state
        return s_next;
    }
    double E() {
        // Implement the energy function here
    };
};

pair<double, state> simAnneal() {
    state s = state();
    state best = s;
    double T = 10000; // Initial temperature
    double u = 0.995; // decay rate
    double E = s.E();
    double E_next;
    double E_best = E;
    mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
    while (T > 1) {
        state next = s.next();
        E_next = next.E();
        if (P(E, E_next, T, rng)) {
            s = next;
            if (E_next < E_best) {
```

```

        best = s;
        E_best = E_next;
    }
    E = E_next;
}
T *= u;
}
return {E_best, best};
}

```

How to use:

Fill in the state class functions as appropriate. If you are trying to find a global maxima and not a minima, ensure that the $E()$ function returns negative of the function you are maximizing and print $-E_{best}$ in the end. Set the below parameters as per your need.

Parameters

- T : Initial temperature. Set it to a higher value if you want the search to run for a longer time.
- u : Decay. Decides the rate of cooling. A slower cooling rate (larger value of u) usually gives better results, at the cost of running for a longer time. Ensure $u < 1$.

The number of iterations the loop will run for is given by the expression

$$N = \lceil -\log_u T \rceil$$

Tips for choosing T and u : If there are many local minimas and a wide state space, set $u = 0.999$, for a slow cooling rate, which will allow the algorithm to explore more possibilities. On the other hand, if the state space is narrower, $u = 0.99$ should suffice. If you are not sure, play it safe by setting $u = 0.998$ or higher. Calculate the time complexity of a single iteration of the algorithm, and use this to approximate a value of N which will prevent TLE, then use the below formula to obtain T .

$$T = u^{-N}$$

Example implementation for TSP

```

class state {
public:
    vector<pair<int, int>> points;
    std::mt19937 mt{ static_cast<std::mt19937::result_type>(
        std::chrono::steady_clock::now().time_since_epoch().count()
    ) };
    state() {
        points = {{0,0},{2,2},{0,2},{2,0},{0,1},{1,2},{2,1},{1,0}} ;
    }
    state next() {
        state s_next;
        s_next.points = points;
        uniform_int_distribution<> choose(0, points.size()-1);
    }
}

```

```

        int a = choose(mt);
        int b = choose(mt);
        s_next.points[a].swap(s_next.points[b]);
        return s_next;
    }

    double euclidean(pair<int, int> a, pair<int, int> b) {
        return hypot(a.first - b.first, a.second - b.second);
    }

    double E() {
        double dist = 0;
        int n = points.size();
        for (int i = 0; i < n; i++)
            dist += euclidean(points[i], points[(i+1)%n]);
        return dist;
    };

    int main() {
        pair<double, state> res;
        res = simAnneal();
        double E_best = res.first;
        state best = res.second;
        cout << "Length of shortest path found : " << E_best << "\n";
        cout << "Order of points in shortest path : \n";
        for(auto x: best.points) {
            cout << x.first << " " << x.second << "\n";
        }
    }
}

```

Further modifications to the algorithm:

- Add a time based exit condition to the while loop to prevent TLE
- The decay implemented above is an exponential decay. You can always replace this with a decay function as per your needs.
- The Probability acceptance function given above, prefers accepting states which are lower in energy because of the $E_{next} - E$ factor in the numerator of the exponent. You can simply remove this factor, to make the PAF independent of the difference in energies.
- The effect of the difference in energies, $E_{next} - E$, on the PAF can be increased/decreased by increasing/decreasing the base of the exponent as shown below:

```

bool P(double E, double E_next, double T, mt19937 rng) {
    double e = 2; // set e to any real number greater than 1
    double prob = pow(e, -(E_next-E)/T);
    if (prob > 1)
        return true;
    else {
        bernoulli_distribution d(prob);
        return d(rng);
    }
}

```

